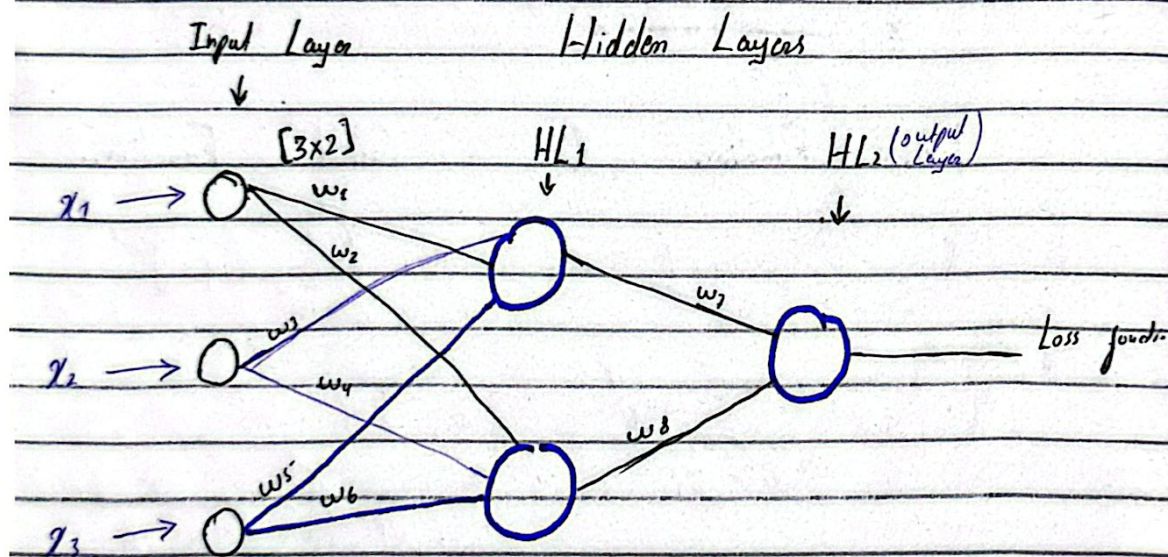


⇒ Backward Propagation & Weight Update Formula:

⇒ Two Layered Neural Network

Id	study hr	Phys hr	Output
95	4	4	1
100	5	2	1
95	2	7	0



* Two Neurons in HL₁ (one Layer can have as many neurons)

* Each input connects to all the neurons in Next Layer ⇒ i.e. x_1 is connected to both neurons of HL₁ and x_2 is connected to both neurons in HL₁

⇒ Weight Update Formula:

* In backward propagation we go from end to start (Right to left "←")

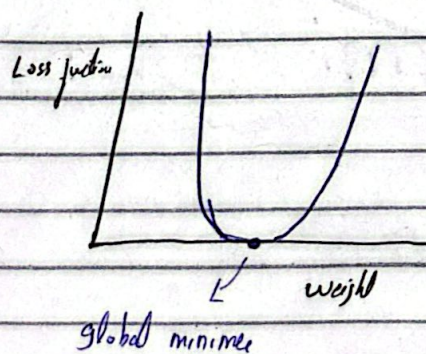
* 1st we will update w_7 & w_8 .

$$\Rightarrow w_{7 \text{ New}} = w_{7 \text{ old}} - \eta \frac{\partial \text{Loss}}{\partial w_{7 \text{ old}}}$$

∴ η = Learning Rate

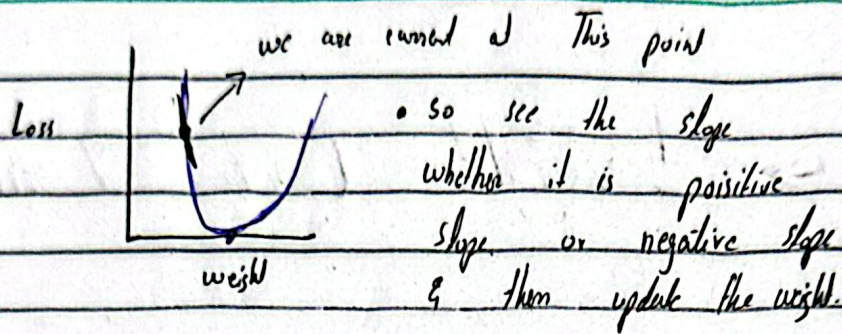
∴ $\frac{\partial \text{Loss function}}{\partial w_{7 \text{ old}}} \Rightarrow \text{slope}$

* By using Loss function we usually get gradient descent.



Note: See
Linear Regression
for gradient
descent & slope
revision

* We need to arrive till global minimum where slope = 0
& when slope is zero means we don't
need to update the weight.



$$W_{\text{New}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W_{\text{old}}}$$

→ Generic Formula:

$$W_{\text{New}} = W_{\text{old}} - \eta \frac{\partial \text{Loss}}{\partial W_{\text{old}}}$$

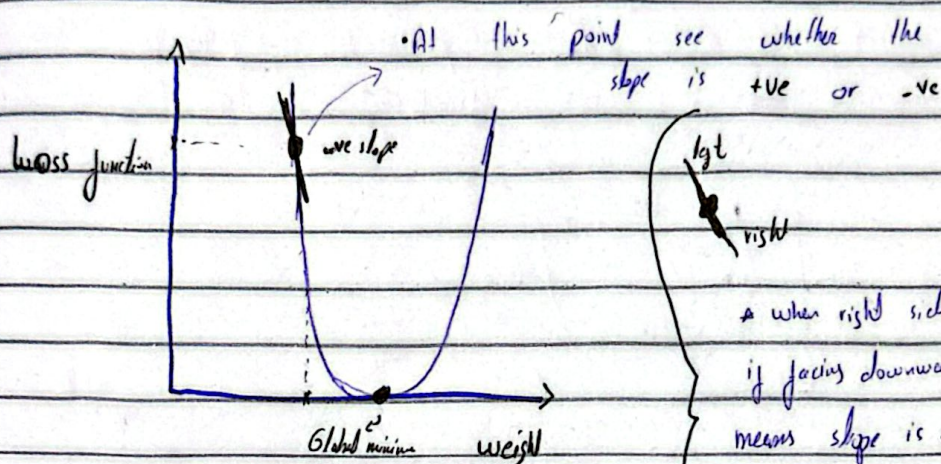
⇒ Optimizer :

* Optimizer are used to reduce the loss value :

* There are different optimizers & we will use Gradient Descent optimizer.

Notes: See in depth in depth

* Suppose at "w = some value" we got
"Loss = some value" we plot it.



$$As: W_{new} = W_{old} - \eta (\text{slope})$$

as slope is -ve

$$So: W_{new} = W_{old} - \eta (-ve)$$

$$\underbrace{-ve \times -ve}_{+ve} = +ve$$

So

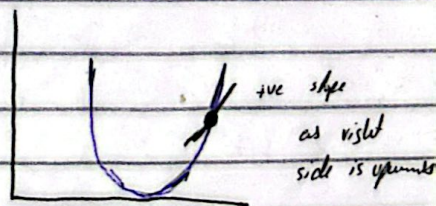
$$W_{new} = W_{old} + \eta (+ve)$$

** Whenever Slope is -ve we need to increase the weight.

$$So: W_{new} = W_{old} - \eta (+ve)$$

$$\underbrace{-ve \times +ve}_{-ve} = -ve$$

$$W_{new} = W_{old} - \eta (-ve)$$

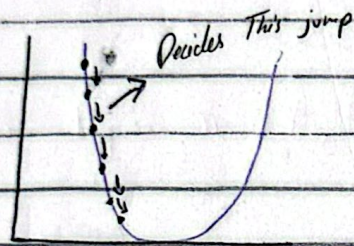


** When Slope is +ve, Decrease The Weight.

→ Learning Rate:

η : learning rate

⇒ it decides the step size (jump) the we took on gradient descent.



* So we should always keep this small as to step or jump smaller.

* if it very large we may skip the global minima in our jump.

x — x

→ When W reaches global minima
then $W_{\text{new}} = W_{\text{old}}$

as at global minima slope equal 0
so

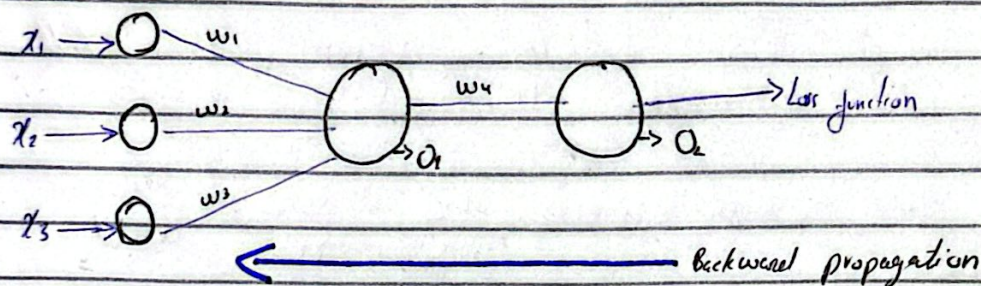
$$\Rightarrow W_{\text{new}} = W_{\text{old}} - \eta(0)$$

$$\Rightarrow W_{\text{new}} = W_{\text{old}}$$

*** Now if I want to update weights of previous layer of " w_2 & w_3 " that is from " $w_1 - w_6$ " we will use "chain rule" as they depends on " w_2 & w_3 " -

⇒ Just To get the concept I will draw another a bit network

↳ Chain Rule of Derivative in Backward propagation



★ To update w_4 :

$$\Rightarrow w_{4, \text{new}} = w_{4, \text{old}} - \eta \left[\frac{\partial \text{Loss}}{\partial w_{4, \text{old}}} \right]$$

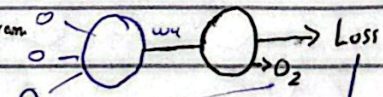
⇒ Let's expand this

$$\frac{\partial \text{Loss}}{\partial w_{4, \text{old}}}$$

⇒

From above diagram

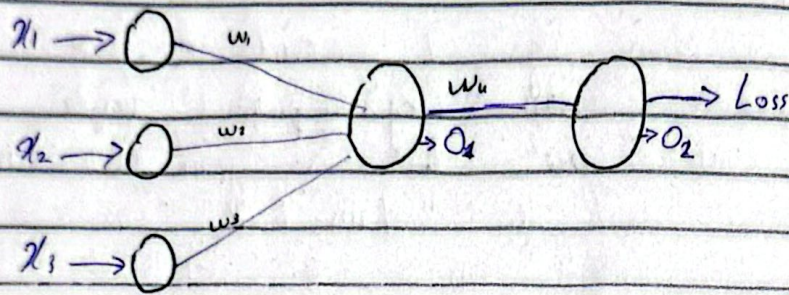
[Right side depends on left]



• Loss is dependent on O_2

O_2 is dependent on w_4

$$\star \star \frac{\partial \text{Loss}}{\partial w_{4, \text{old}}} = \frac{\partial \text{Loss}}{\partial O_2} \times \frac{\partial O_2}{\partial w_4}$$



* In Expansion Right Side "Variable" depends on its Left side variable.

→ e.g. "Loss" (on right side) depends on " O_2 " (to its left side)

→ " O_2 " (right side Now) depends on " w_4 " (to its left).

So: $\frac{\partial \text{Loss}}{\partial w_{4 \text{ old}}}$ Means from Loss Till w_4

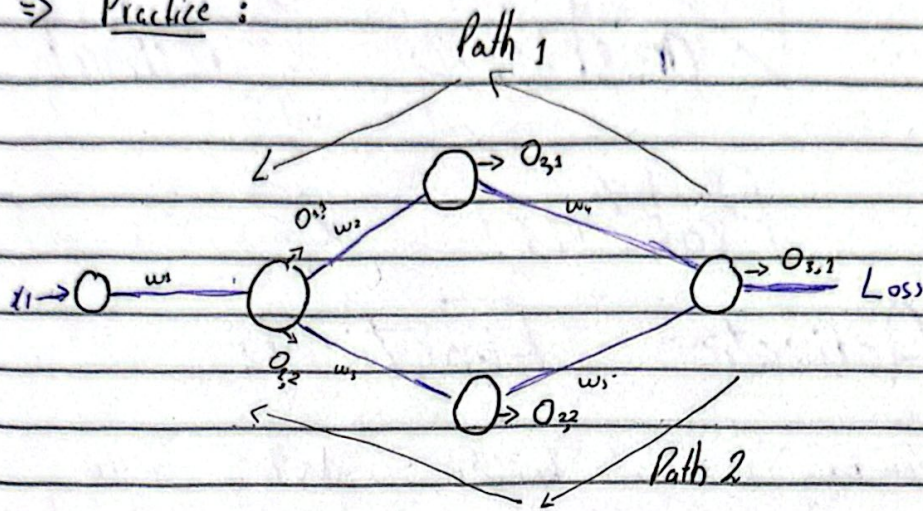
So expansion becomes $\Rightarrow \frac{\partial \text{Loss}}{\partial w_{4 \text{ old}}} = \frac{\partial \text{Loss}}{\partial O_2} * \frac{\partial O_2}{\partial w_{4 \text{ old}}}$

$$w_{1 \text{ New}} = w_{1 \text{ old}} - \eta \left[\frac{\partial \text{Loss}}{\partial w_{1 \text{ old}}} \right]$$

→ Expanding This (From Loss Till w_1)

$$\frac{\partial \text{Loss}}{\partial w_{1 \text{ old}}} = \frac{\partial \text{Loss}}{\partial O_2} * \frac{\partial O_2}{\partial O_1} * \frac{\partial O_1}{\partial w_{1 \text{ old}}}$$

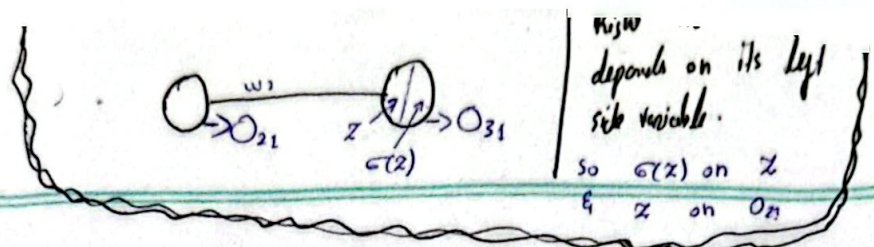
⇒ Practice :



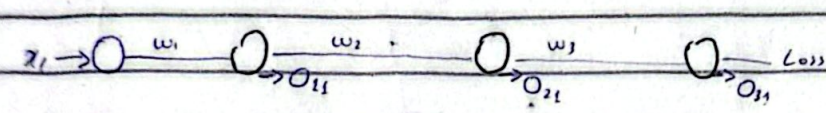
$$w_{1_{\text{new}}} = w_{1_{\text{old}}} - \eta \left[\frac{\partial \text{Loss}}{\partial w_{1,1}} \right]$$

To expand this see There are Two paths
so 1st we will expand one path & then
another & add both

$$\frac{\partial \text{Loss}}{\partial w_{1,1}} = \left[\frac{\partial \text{Loss}}{\partial O_{3,1}} * \frac{\partial O_{3,1}}{\partial z_1} * \frac{\partial z_1}{\partial w_{1,1}} \right] + \left[\frac{\partial \text{Loss}}{\partial O_{3,1}} * \frac{\partial O_{3,1}}{\partial z_2} * \frac{\partial z_2}{\partial w_{1,1}} \right]$$



→ okay let's get/focus on " $\frac{\partial O_{31}}{\partial O_{21}}$ "



"observe in graph To get O_{31} what we need to do?"

⇒ ⇒ • To get O_{31} we need to
1st, in step 1 find z
then in step 2 apply activation
function on z . (Note: See ANN pages to revise)

So ⇒ $z = O_{21} * w_3 + b$
Activation (z)

• As we are using sigmoid
so $\sigma(z)$

* So we can say The input that is going to "Activation Func" is " z ."

$$O_{31} = \sigma(w_3 * O_{21} + b)$$

or
 $O_{31} = \sigma(z)$

*** So for " $\sigma(z)$ " input is " z " means $\sigma(z)$ is on "right side" & " z " is to its left

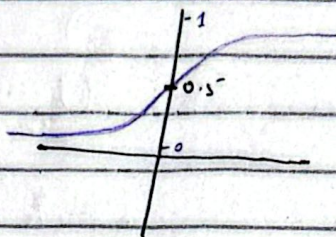
*** & for " z " input is O_{21} .
⇒ Means " $\sigma(z)$ " depends on " z "
& " z " depends on " O_{21} "

[Note: See Diagram above the Page.]

so: $\frac{\partial O_{31}}{\partial O_{21}} = \frac{\partial (\sigma(z))}{\partial (z)} * \frac{\partial (z)}{\partial O_{21}}$

⇒ Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

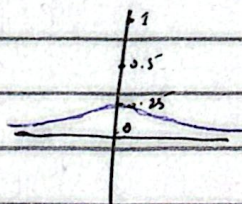


* sigmoid ranges: 0 - 1

* but derivatives of sigmoid ranges: 0 - 0.25

$$\sigma(z) \Rightarrow 0 - 1$$

$$\sigma'(z) \Rightarrow 0 - 0.25$$



$$\Rightarrow \frac{\partial O_{31}}{\partial O_{21}} = \frac{\partial (\sigma(z))}{\partial (z)} * \frac{\partial (z)}{\partial O_{21}}$$

$$= \frac{\partial (\sigma(w_3 * O_{21} + b_3))}{\partial O_{21}}$$

$$\frac{\partial O_{31}}{\partial O_{21}} \leq 0 \leq \sigma(z) \leq 0.25 * w_3$$

• after derivative We will get this
but This is a smaller
values i.e 0 - 0.25

Now:

$$\frac{\partial \text{Loss}}{\partial w_1} = \frac{\partial \text{Loss}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial w_1}$$

$\downarrow$ $\downarrow$ $\downarrow$
 small value small value small value
 0-0.25 0-0.25 0-0.25

- for each we will get small value & we are multiplying it, so it will become even more smaller

$$\frac{\partial \text{Loss}}{\partial w_1} \Rightarrow \text{Very small value}$$

$$w_{1_{\text{new}}} = w_{1_{\text{old}}} - \eta \left[\frac{\partial \text{Loss}}{\partial w_{1_{\text{old}}}} \right]$$

$\downarrow$ $\rightarrow$ very small value
 already small

$\rightarrow$ & we are again multiplying two very small values i.e. " η " & " $\frac{\partial \text{Loss}}{\partial w_{1_{\text{old}}}}$ ", we will get extreme small value.

So,

$$w_{1_{\text{new}}} = w_{1_{\text{old}}} - (\text{extremely small value})$$

* * This will not affect the weight as much & both w_{new} & w_{old} are almost equal, if weight not change we will not reach global minima.

$$\Rightarrow w_{1_{\text{new}}} \approx w_{1_{\text{old}}}$$

* In deep Neural Network if we use sigmoid we will get stuck. (Fine in smaller Networks)

* To solve this problem researchers explored other activation functions i.e.

(1) ReLU (2) Tanh (3) PReLU (4) Swish.

X ——— X